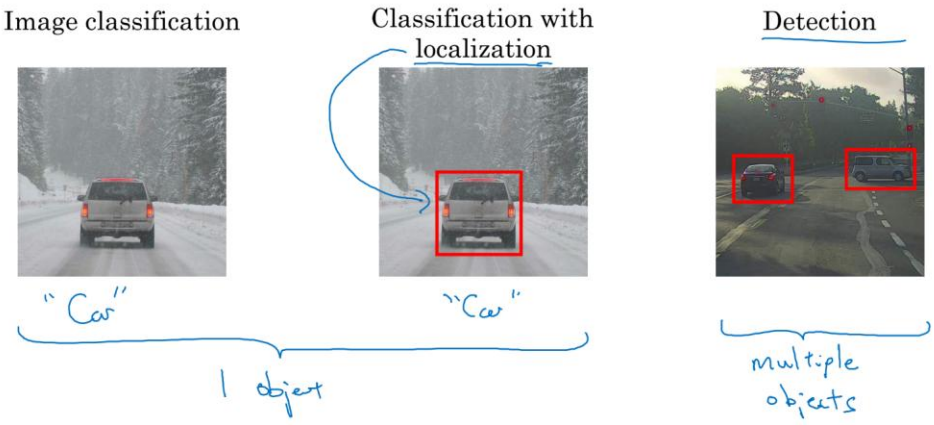
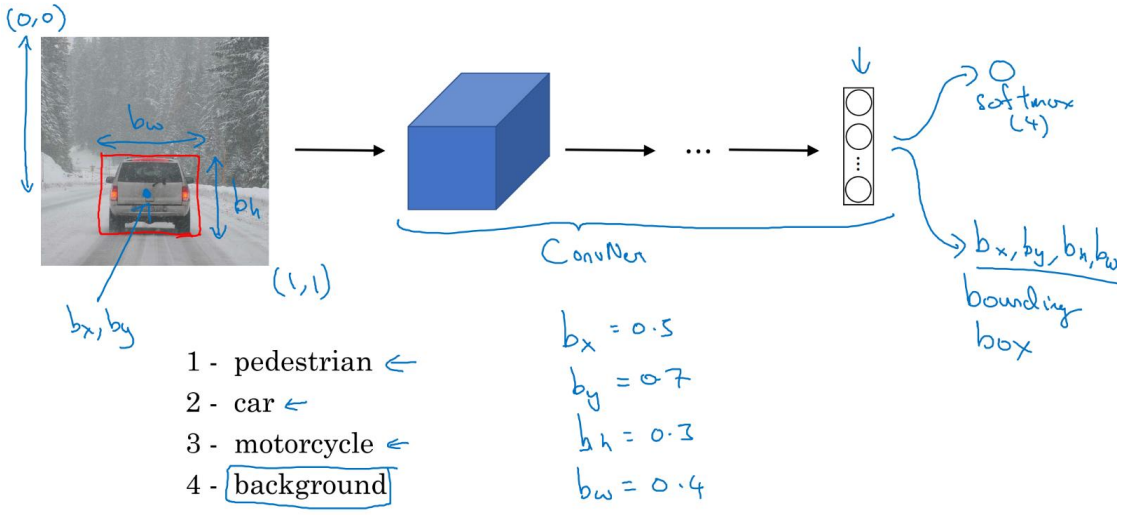
Objection Localization

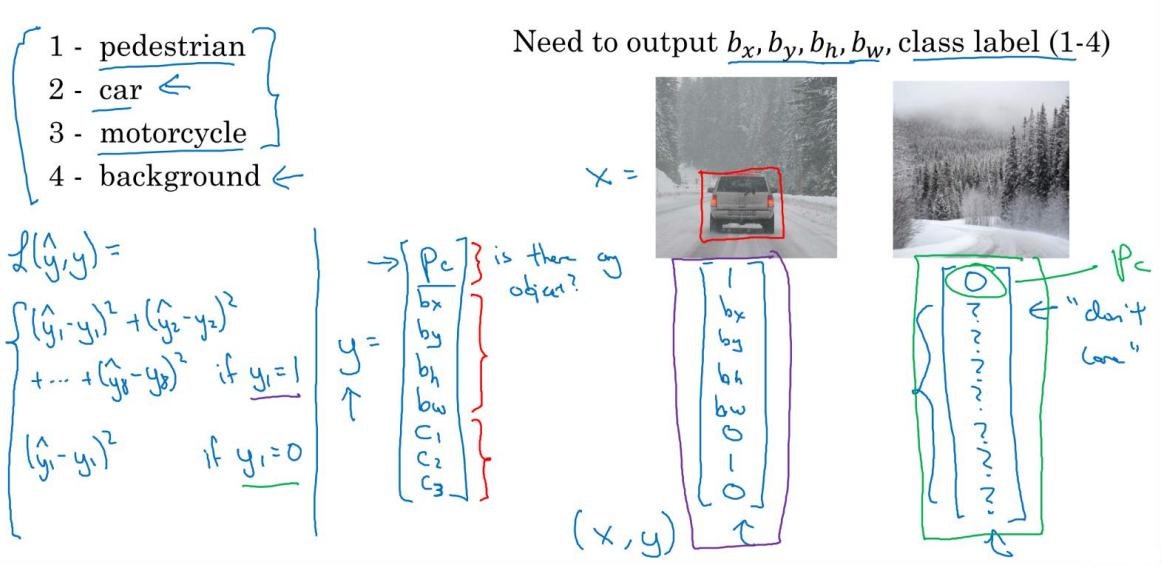
- **Localization**: putting a bounding box around the classified object
- **Detection**: detect and localize multiple objects in a picture



- Classification with localization
  - Softmax with 4 possible outputs (class of object)
  - Another output: 4 units of **bounding box** (b_x, b_y, b_H, b_W).
  - (b_x, b_y,) is the mid-point, and b_H, b_W are the height and width of the bounding box

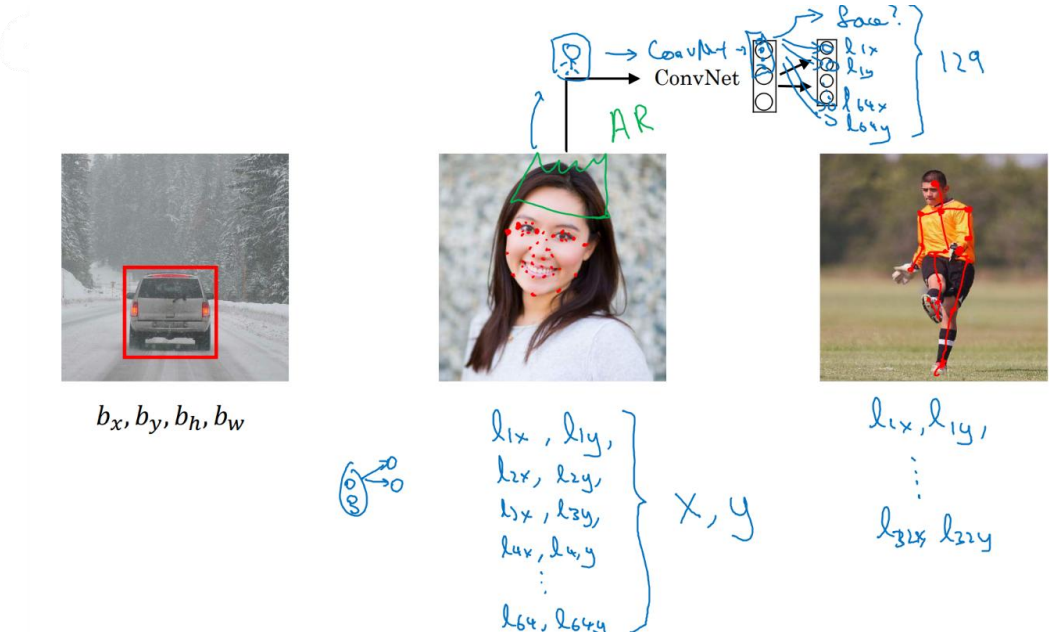


- Defining the target label y
  - Output y:
    - First component p_c: is there an object?
    - If there is an object, output the params of the bounding box b_x, b_y, b_H, b_W
    - If there is an object, output which class c_1, c_2, c_3 (one-hot encoded) (assuming at most one object here)
    - If no object, we don't care about the bounding box and class, so ? in y
  - Loss: $\Sigma (\hat{y}_i - y_i)^2$ (i = length of y) if $y_1 = 1$, or just $(\hat{y}_1 - y_1)^2$ if $y_1 = 0$
  - In practice, use log-like loss for c_1, c_2, c_3 to the softmax output, squared error for bounding box coordinates, and logistic regression loss for p_c



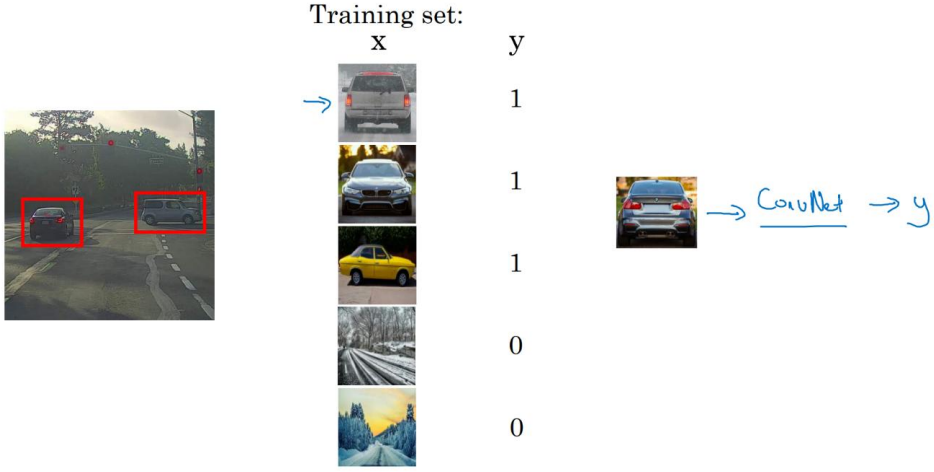
Landmark Detection

- In more general cases, you can have the NN just output x and y coordinates of important points in the image (**landmarks**)
- Landmark coordinates { l_{ix}, l_{iy} } → x, y = set of images and landmark annotations
- ConvNet outputting whether there is a face or not and coordinates of landmarks
- Landmark I needs to be consistent between images

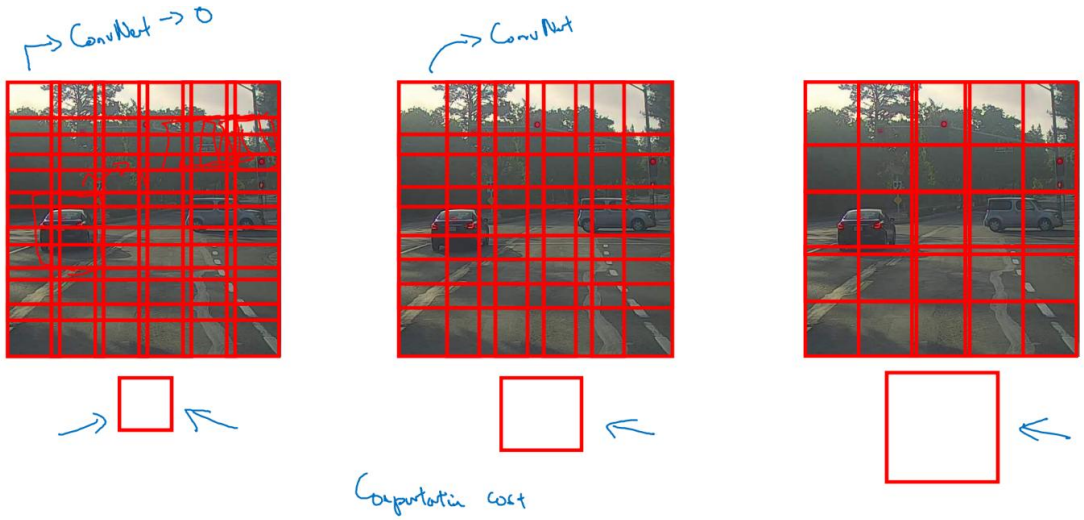


Object Detection

- Start by creating a labeled training set with closely cropped examples. Then create a ConvNet that take one of these images as input and outputs y (0 or 1, car or not)

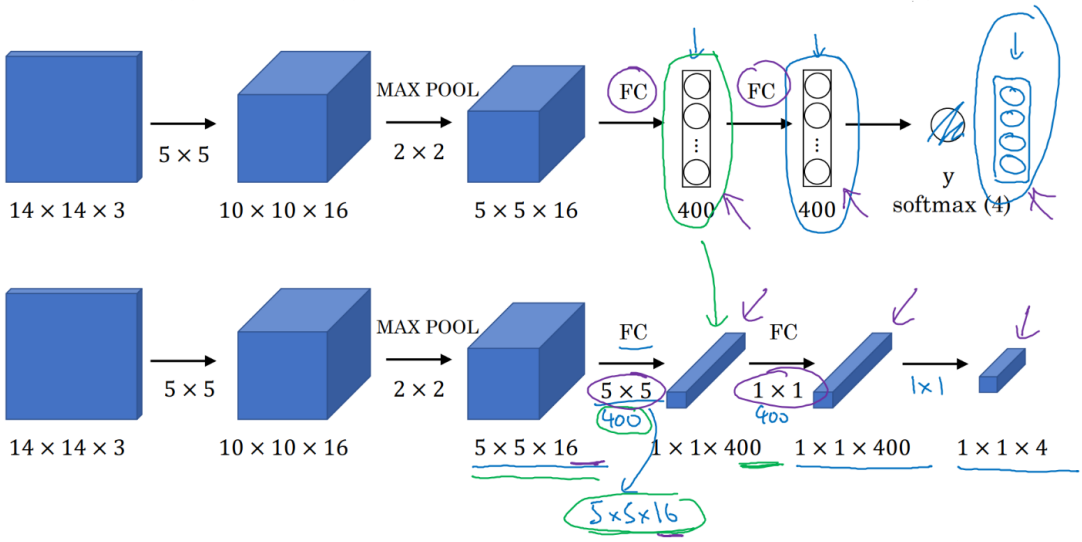


- **Sliding windows detection algorithm**
  - Pick a certain window size. Input into ConvNet rectangular regions of that size (shifted across/down the whole image) and have it make predictions for each
  - If there is a car in the image, there will be a window where ConvNet will detect it
  - Disadvantage: computational cost of so many images
  - Stride will reduce # windows, but coarser granularity may hurt performance
  - Infeasibly slow or unable to localize accurately
  - Sliding window can be implemented convolutionally much more efficiently

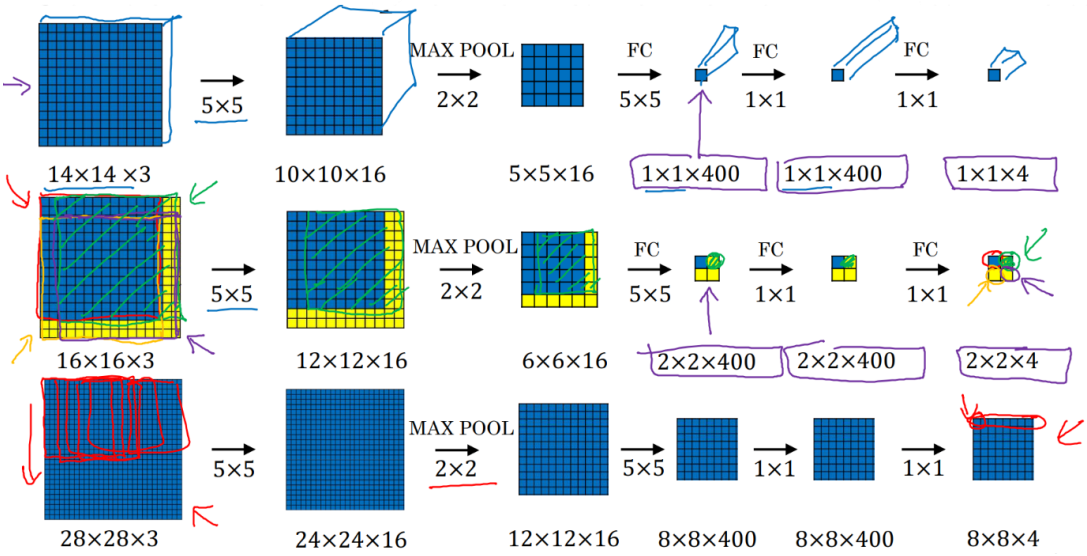


Convolutional Implementation of Sliding Windows

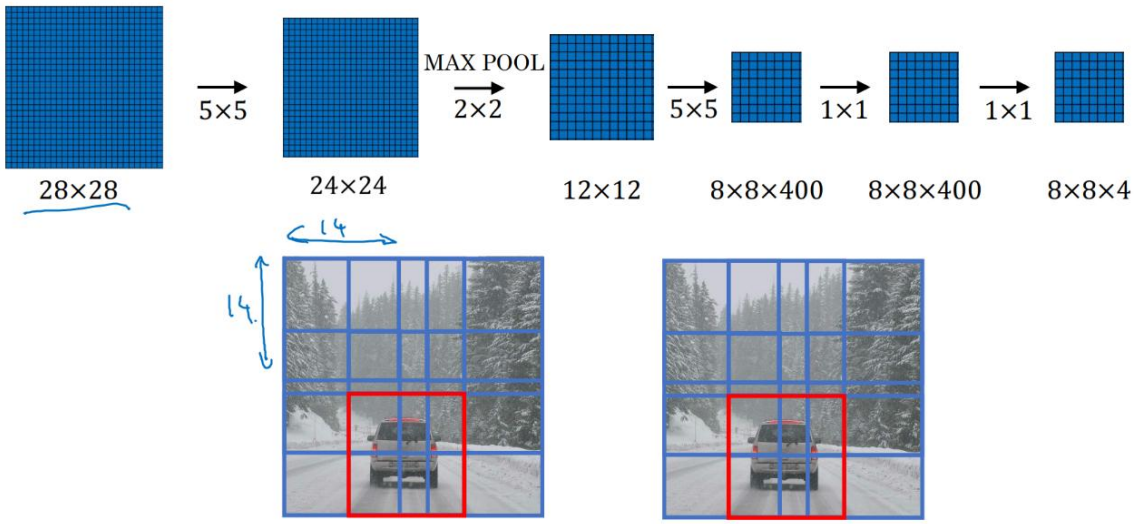
- Turning FC layer into convolutional layers
    - Softmax outputs probability of each class here
    - Convolve with 400 5x5x16 filters → 1x1x400. Mathematically the same as a FC layer: each of the 400 nodes has a 5x5x16 filter so is some arbitrary linear function of the 5x5x16 activations from the previous layer
    - Then convolve with a 1x1, which gives the same 1x1x400 dimensions
    - One last 1x1 filter and softmax activation down to 1x1x4



- Convolution implementation of sliding windows
    - If ConvNet inputs 14x14x3 images but test set image is 16x16x3 (yellow stripe)
    - Slide around by 2 to get all regions (top-left, top-right, bottom-left, bottom-right)
    - A lot of the computation done by the four ConvNets is highly duplicative
    - Conv sliding windows combines the forward passes of the ConvNet to share computation. Now you get a 2x2x400 intermediate volume (instead of 1x1x400) and a final output that is 2x2x4 (instead of 1x1x4). Each position of the 2x2 gives the result for the corresponding cube of image (top-left, top-right, bottom-left, bottom-right)

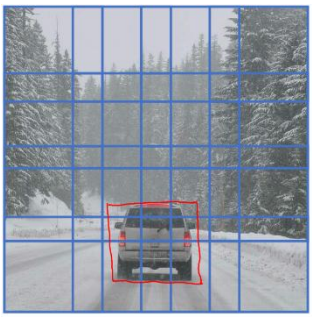


- Instead o sequentially, implement entire image and convolutionally make all predictions at the same time by one forward pass through the ConvNet



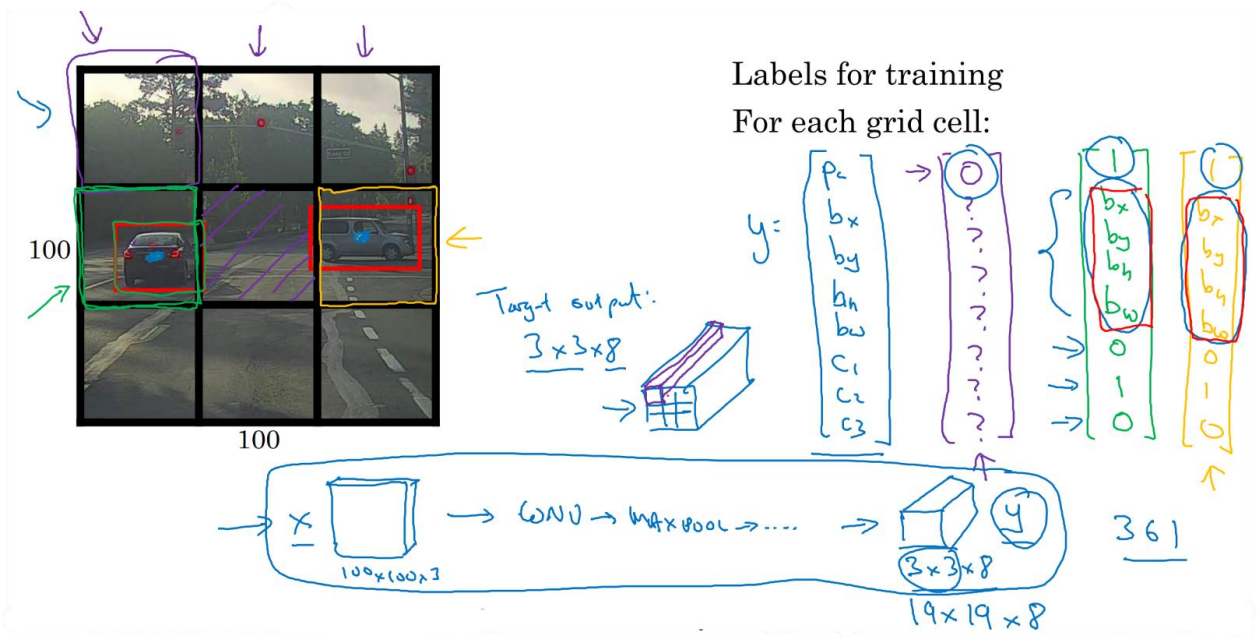
- Weakness: position of bounding boxes will not be too accurate

Bounding Box Predictions

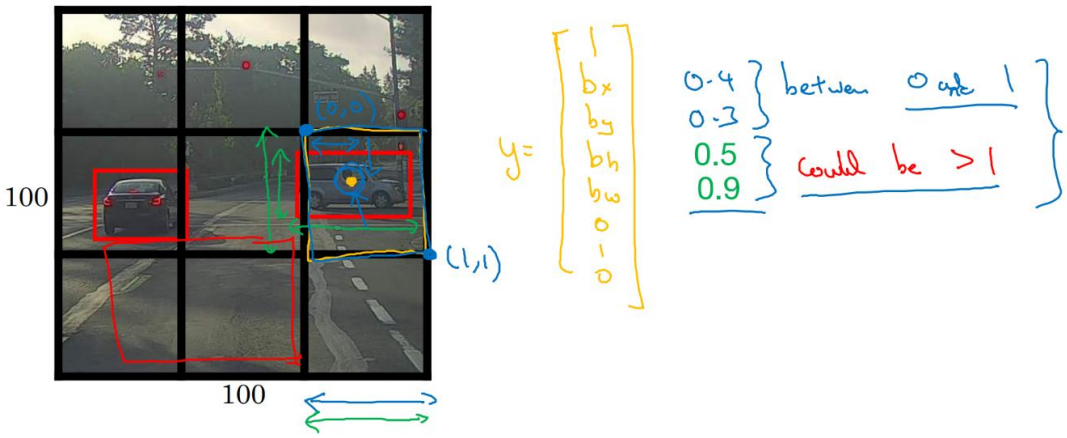


- Output accurate bounding boxes
  - Perfect bounding box may not even be a perfect square

- **YOLO (You Only Look Once) algorithm**
  - Place a grid on image (usually finer, like 19x19) and apply image classification and localization algorithm to each cell of the grid. For each grid cell, specify a label $y = [p_c, \text{bounding params, classes}]^T$
  - Assigns each object to the grid cell containing the midpoint (doesn't count some sticking into the next cell)
  - Final output volume is 3x3x8 (8D y vector)
  - Train network on x 100x100x3. Choose conv and maxpool layers such that it eventually maps to a 3x3x8 volume
  - Works okay as long as you don't have more than one object per cell (finer grid reduces chance of multiple objects per cell)
  - NN outputs bounding boxes of any aspect ratio with much more precise coordinates that aren't just dictated by the stride size of sliding windows classifier. Run the algorithm all at once, very efficient

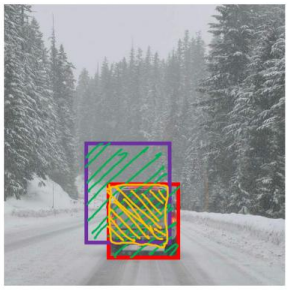


- Specify the bounding boxes
  - $0 < b_x, b_y < 1$ (orange dot is within bounds of grid it's assigned to)
  - b_H, b_W fraction of grid cell, can be > 1 (if bounding box spills into another cell)



Intersection Over Union (IOU)

- Used for evaluating and adding another component to an object detection algorithm
- Evaluating object localization
  - Ratio of areas (intersection / union)
  - Perfect = 1, 0.5 is most common, can be more stringent
  - Generally, IoU is a measure of the overlap between two bounding boxes



Intersection over Union

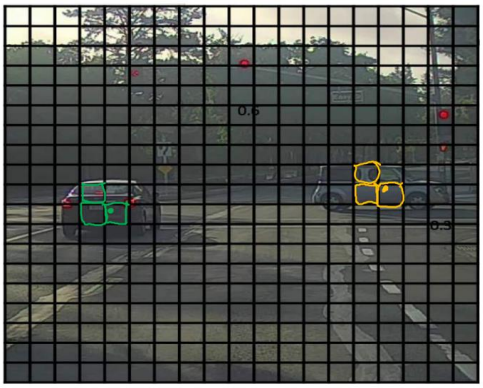
$$= \frac{\text{size of}}{\text{size of}}$$

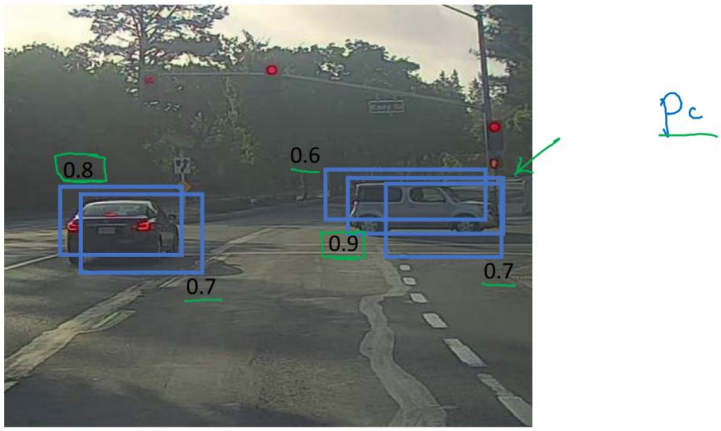
"Correct" if IoU ≥ 0.5 ←

0.6 ←

Non-Max Suppression

- Non-max suppression ensures the algorithm only detects a given object once
- Example
    - Only one grid cell has the midpoint here, but in practice you're running an object classification and localization algorithm for every split cell. Several cells might think they have the center of the car in it. Non-max suppression cleans this.
    - Takes largest p_c, suppresses any of the rest with high overlap (IoU). Find the next highest p_c (likely a different object) and repeat. These are the final predictions

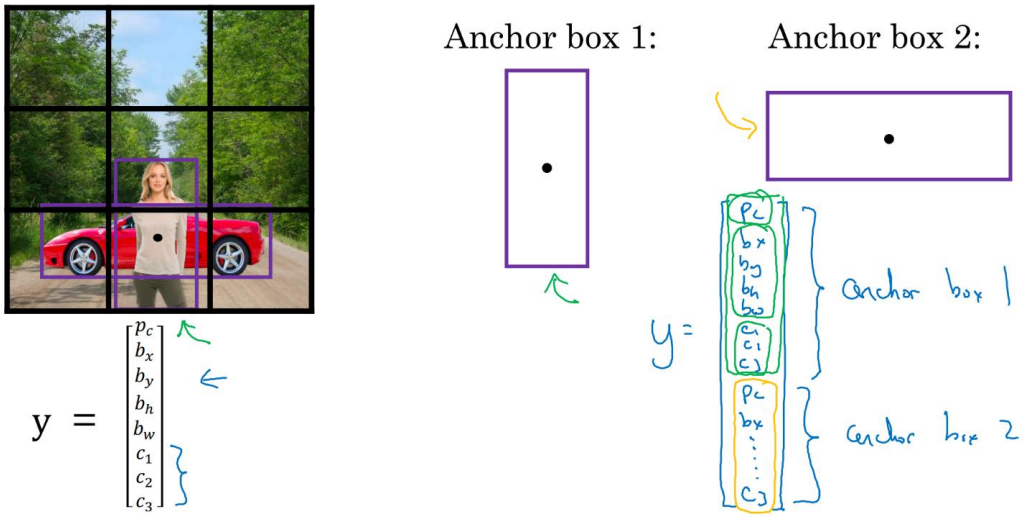




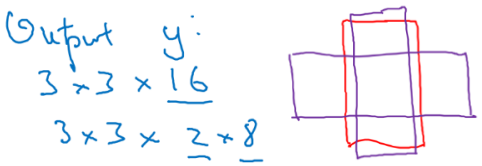
- Each (cell) output prediction is $[p_c\ b_x\ b_y\ b_H\ b_W]^T$
    - looking at one object class here. If you have more clases, independently carry out non-max suppression c times, one on each of the output classes
- Discard all boxes with $p_c \leq 0.6$.
- While there are any remaining boxes:
    - Pick the box with the largest pc → output that as a prediction
    - Discard any remaining box with IoU ≥ 0.5 with the box output in the previous step

Anchor Boxes

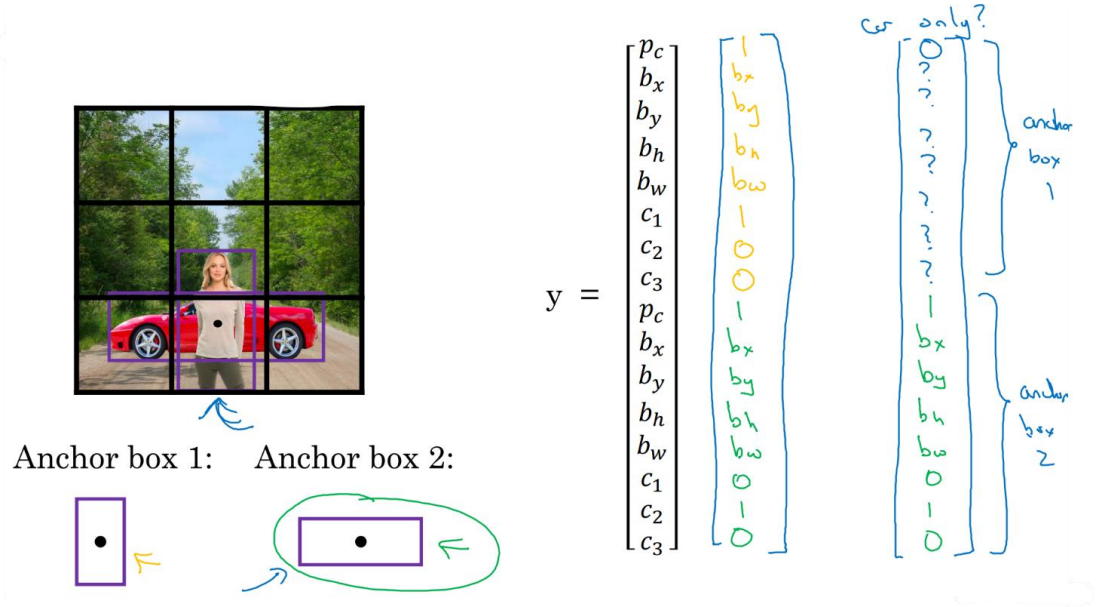
- What is a grid cell wants to detect multiple objects?
- Overlapping objects
    - Midpoint of pedestrian and car are in the same place
    - y vector can't output two detections, would have to pick one
    - Predefine different shapes (anchor boxes) and associate predictions with each
    - Pedestrian shape is similar to anchor box 1, car is similar to anchor box 2
    -



- Previously: Each object in training image is assigned to grid cell that contains that object's midpoint (output y: 3x3x8)
- With two anchor boxes: Each object in training image is assigned to grid cell that contains object's midpoint and anchor box for the grid cell with highest IoU
    - See which of the anchor boxes has a higher IoU with the ground truth bounding box. Whichever it is, that inject then gets assigned (grid cell, anchor box) pair



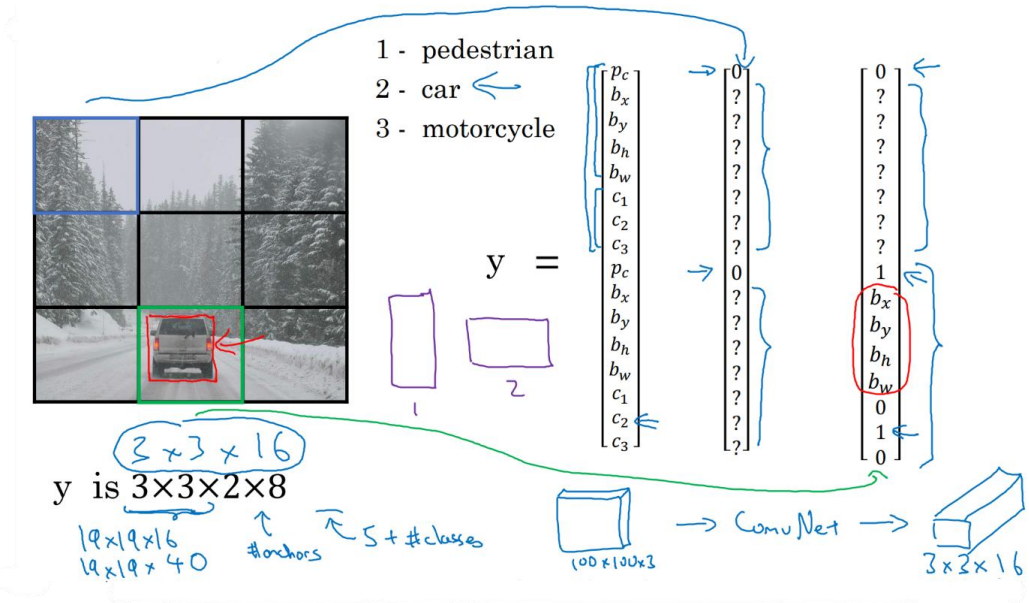
- Example
  - Pedestrian resembles anchor box 1 → assigned to top half of y vector, c1=0
  - Car resembles anchor box 2 → assigned to bottom half of y vector, c2=1
  - Car only: first anchor box is no object [0 ? ? ?]



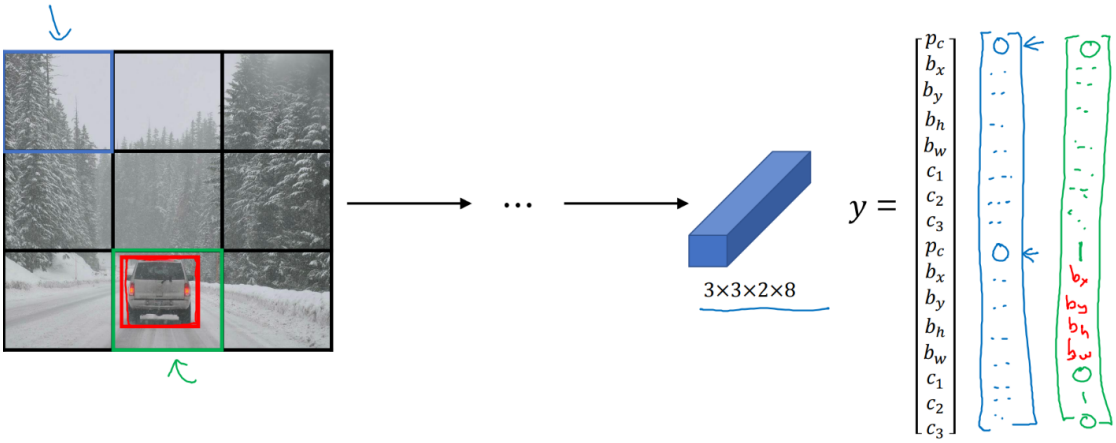
- Can't handle two anchor boxes but three objects in the same cell. Also can't handle two objects in the same cell with the same anchor box shape. You would have to implement a default tiebreaker
- K-means algorithm to group the types of objects shapes and use that to select a set of anchor boxes that is most stereotypically representative of the objects you're trying to detect

Putting It Together: YOLO Algorithm

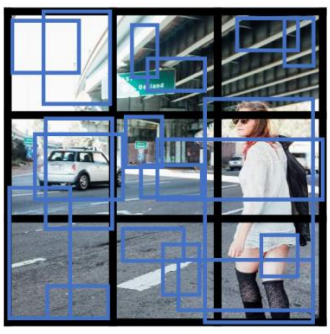
- Training
  - y is grid dimensions x # anchors x (5 {p_c, b's} + # classes)
  - Train a ConvNet that takes a 100x100x3 image and outputs volume 3x3x16



- Making predictions

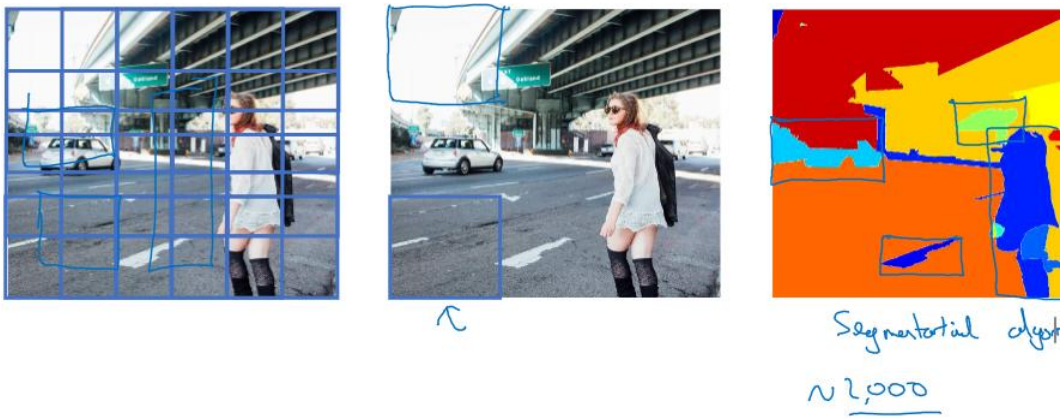


- Outputting the non-max suppressed outputs
  - For each grid cell, get 2 predicted bounding boxes
  - Get rid of low probability predictions
  - For each class (pedestrian, car, motorcycle) use non-max suppression to generate final predictions



Region Proposals

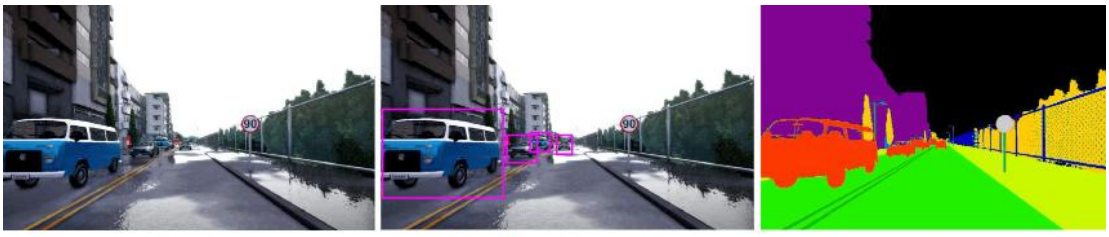
- Used a but less often
- Convolution algorithm classifies a lot of regions where there's clearly no objects
- **Regions with Convolutional Networks** (R-CNN)
    - Tries to pick just a few regions that makes sense to your ConvNet classifier
    - Region proposals via **segmentation algorithm** to figure out what could be objects. Put boxes around blobs and run classifier on just those



- **R-CNN**: Propose regions, classify proposed regions one at a time, output label + bounding box
- **Fast R-CNN**: Propose regions, use convolution implementation of sliding windows to classify all the proposed regions
- **Faster R-CNN**: use convolutional network to propose regions

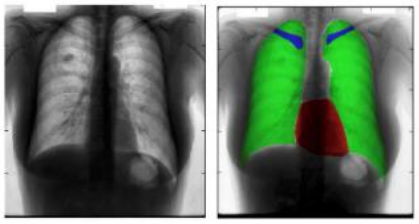
Semantic Segmentation with U-Net

- Draw a careful outline around the object that is detected so that you know exactly which pixel belong to the object
- Object Detection vs. Semantic Segmentation
    - Object detection: boxes around objects
    - Semantic segmentation: label every pixel (e.g. as drivable road, green)

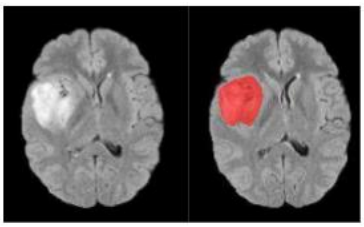


Input image | Object Detection | Semantic Segmentation

- Motivating example for U-Net: diagnosing disease
  - Determining which pixel correspond to certain parts of the patient's anatomy.
  - Segmentation makes it easier to spot irregularities and diagnose diseases. Also helps with planning surgeries

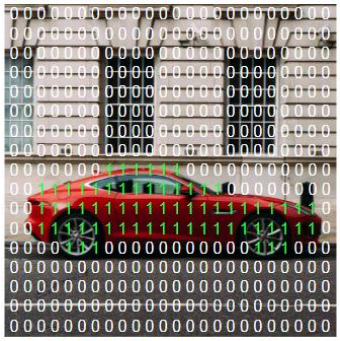


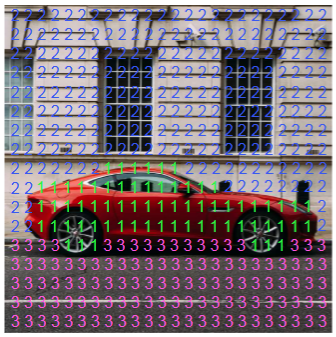
Chest X-Ray



Brain MRI

- Per-pixel class labels
  - Binary classification → U-Net outputs 0 or 1 for every pixel
  - Multiple classification → U-Net matrix of labels 1, 2, …, c for every pixel



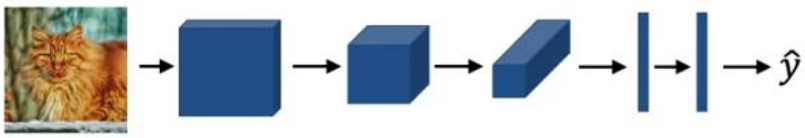


1. Car
2. Building
3. Road

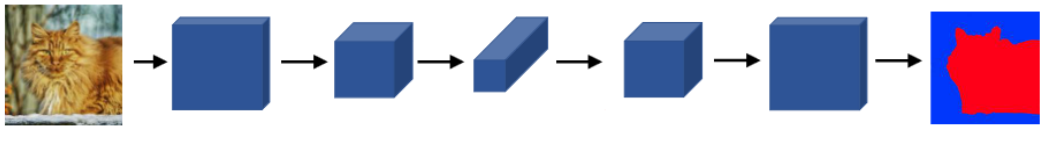
Segmentation Map

- Deep Learning for Semantic Segmentation
  - Modify object recognition network: remove last few layers
  - Dimensions of image need to get bigger so they can gradually blow back up to full-size image for the output
  - Deeper into U-Net → height/width go up, # channels goes down

Obj Recognition:



U-Net:



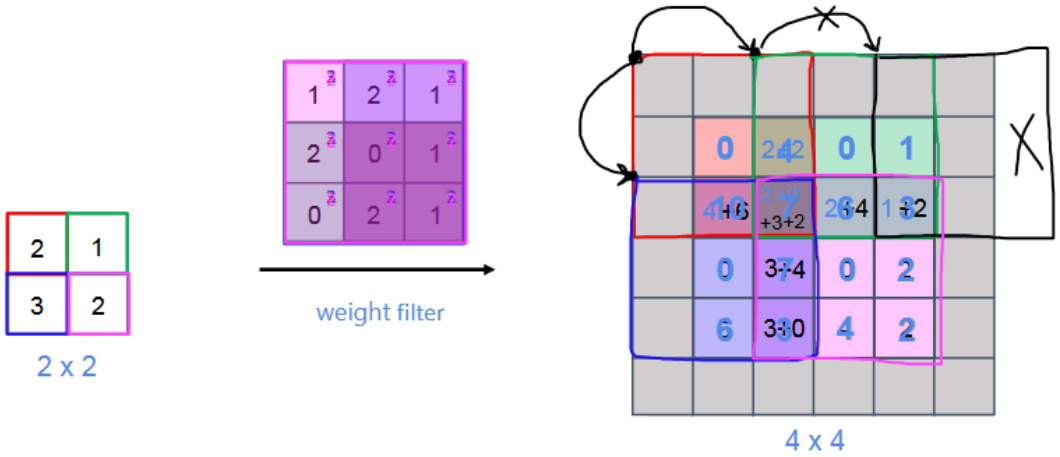
Transpose Convolutions

- Key part of U-Net architecture. Blow up 2x2 input into 4x4 output
- Normal convolution shrinks dimensions but has more channels
- Transpose convolution remains flat, but has wider dimensions

Normal Convolution

Transpose Convolution

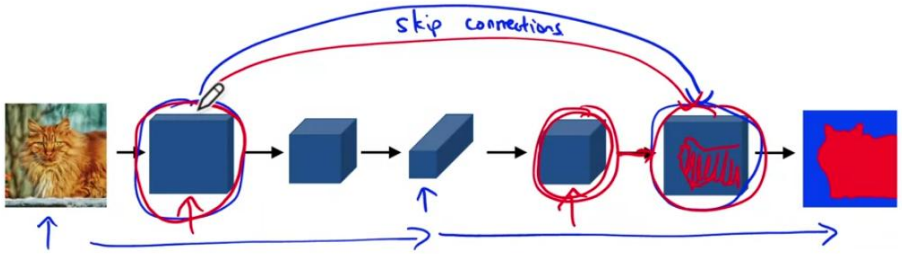
- Instead of placing the filter on the input, you instead place a filter on the output
  - 2 * 3x3 filter → paste 3x3 result in upper left position of output.
  - Padding area won't contain any values, so do not fill in (only use for positioning)
  - 1 * 3x3 filter → paste in output shifted over s=2.
  - When there is some overlap with the previous → add values together
  - Repeat across/down input



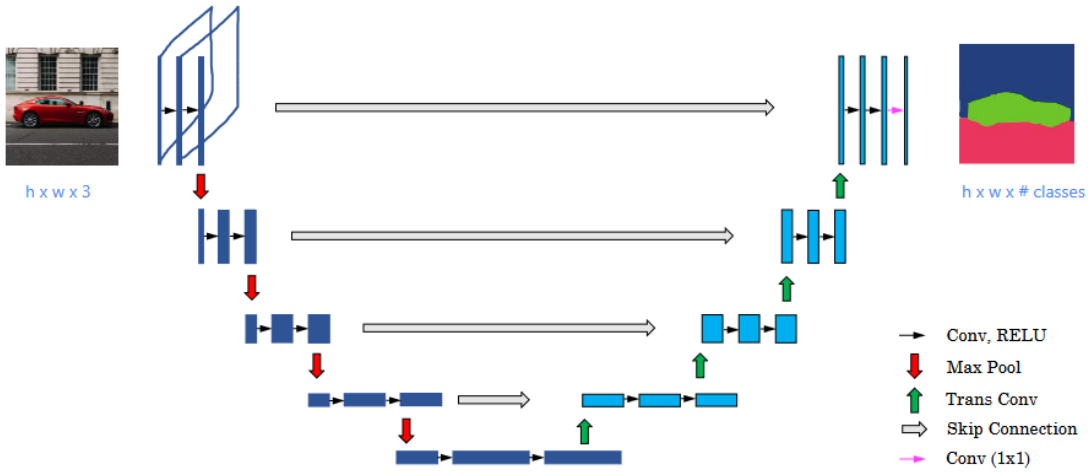
filter f x f = 3 x 3 padding p = 1 stride s = 2

U-Net Architecture

- Intuition: Deep Learning for Semantic Segmentation
  - Normal convolutions for first part of network (compresses image)
  - Second half uses transpose convolution to blow size back up
  - Faster with skip connection. Earlier block of activations is copied directly to later
  - For the final decision layer there are two helpful types of information: high-level spatial/contextual info from previous layer and detailed fine-grain spatial info



- Architecture Diagram
  - Looking at input edge-on (volume behind) for ease of visualization
  - First part of the U-Net uses normal feed forward NN convolutional layers
  - Black arrow = conv layer follow by a ReLU activation function
  - Red arrow = max pooling layer (reduction of height and width)
  - Height and width at bottom of U shape is very small
  - Green arrow = transpose convolutional layer
  - Gray arrow = skip connection (copy activations over to the right)
  - Back to activation set that is the original input image's height and width (h x w)
  - More normal feedforward convolutions, followed by 1x1 conv (pink arrow)
  - Final ouput is h x w x $n_{classes}$
  - Argmax over n classes to classify each pixel into one of the classes



You are building a 3-class object classification and localization algorithm. The classes are: pedestrian (c=1), car (c=2), motorcycle (c=3). What should y be for the image below? Remember that "?" means "don't care", which means that the neural network loss function won't care what the neural network gives for that component of the output. Recall $y = [p_c, b_x, b_y, b_h, b_w, c_1, c_2, c_3]$.

$\boldsymbol{y} = [\mathbf{0}, ?, ?, ?, ?, ?, ?, ?, ?]$

$y = [1, ?, ?, ?, ?, 0, 0, 0]$

$y = [1, ?, ?, ?, ?, ?, ?, ?, ?]$

$y = [?, ?, ?, ?, ?, ?, ?, ?, ?]$

You are working on a factory automation task. Your system will see a can of soft-drink coming down a conveyor belt, and you want it to take a picture and decide whether (i) there is a soft-drink can in the image, and if so (ii) its bounding box. Since the soft-drink can is round, the bounding box is always square, and the soft-drink can always appear the same size in the image. There is at most one soft-drink can in each image. Here are some typical images in your training set:

The most adequate output for a network to do the required task is $y = [p_c, b_x, b_y, b_h, b_w, c_1]$. (Which of the following do you agree with the most?)

True, p_c indicates the position of the object and its bounding box, and c_1 indicates the probability of there being a can of soft-drink.

False, since we only need two values c_1 for no soft-drink can and c_2 for soft-drink can.

False, we don't need b_h, b_w since the cans are all the same size.

True, since this is a localization problem.

With the position b_x, b_y we can completely characterize the position of the object if it is present. We should use only one additional logistic unit to indicate if the object is present or not.

When building a neural network that inputs a picture of a person's face and outputs N landmarks on the face (assume that the input image contains exactly one face), we need two coordinates for each landmark, thus we need 2N output units. True/False?

False **True**

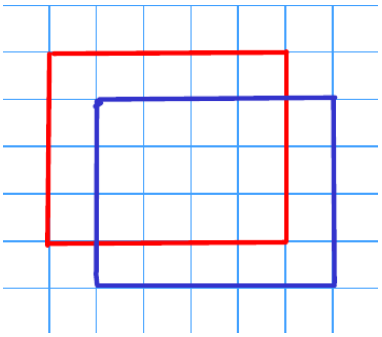
Each landmark is a specific position in the face's image, thus we need to specify two coordinates for each landmark.

When training one of the object detection systems described in the lectures, you need a training set that contains many pictures of the object(s) you wish to detect. However, bounding boxes do not need to be provided in the training set, since the algorithm can learn to detect the objects by itself.

False True

You need bounding boxes in the training set. Your loss function should try to match the predictions for the bounding boxes to the true bounding boxes from the training set.

What is the IoU between the red box and the blue box in the following figure? Assume that all the squares have the same measurements.



$\frac{3}{7}$ $\frac{1}{2}$ $\frac{2}{5}$ $\frac{4}{5}$

IoU = intersection / union = 12 / (12+16) = 3/7

Suppose you run non-max suppression on the predicted boxes below. The parameters you use for non-max suppression are that boxes with probability ≤ 0.4 are discarded, and the IoU threshold for deciding if two boxes overlap is 0.5.

Notice that there are three bounding boxes for cars. After running non-max suppression, only the bounding box of the car with 0.73 is kept from the three bounding boxes for cars. True/False?

True. The non-maximum suppression eliminates the bounding boxes with scores lower than the ones of the maximum.

False. All the cars are eliminated since there is a pedestrian with a higher score of 0.98.

False. Two bounding boxes corresponding to cars are left since their IoU is zero.

One of the bounding boxes for cars is eliminated because it has a lower score and an IoU higher than 0.5.

If we use anchor boxes in YOLO we no longer need the coordinates of the bounding box b_x, b_y, b_h, b_w since they are given by the cell position of the grid and the anchor box selection. True/False?

False True

We use the grid and anchor boxes to improve the capabilities of the algorithm to localize and detect objects, for example, two different objects that intersect, but we still use the bounding box coordinates.

Semantic segmentation can only be applied to classify pixels of images in a binary way as 1 or 0, according to whether they belong to a certain class or not. True/False?

True **False**

The ideas for multi-class classification can be applied to semantic segmentation.

Using the concept of Transpose Convolution, fill in the values of **X**, **Y** and **Z** below.

(*padding = 1, stride = 2*)

Input: 2x2

1	2
3	4

Filter: 3x3

1	1	1
0	0	0
-1	-1	-1

Result: 6x6

	0	0	0	X	
	Y	4	2	2	
	0	0	0	0	
	-3	Z	-4	-4	

X = 0, Y = 2, Z = -1

X = 0, Y = -1, Z = -4

X = 0, Y = 2, Z = -7 Y = 2*1 Z = 3(-1)+4(-1) = -7

X = 0, Y =-1 , Z = -7

When using the U-Net architecture with an input $h \times w \times c$, where c denotes the number of channels, the output will always have the shape $h \times w \times c$. True/False?

False True

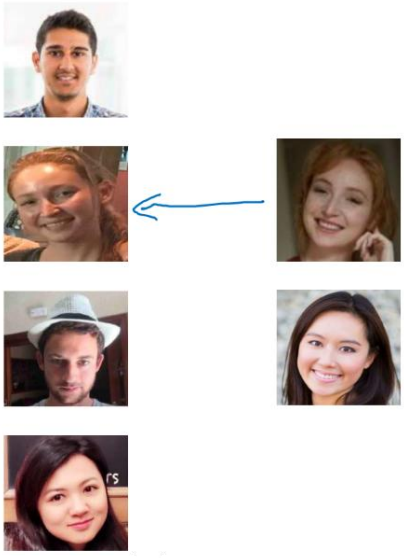
The output of the U-Net architecture can be $h \times w \times k$ where k is the number of classes. The number of channels doesn't have to match between input and output.

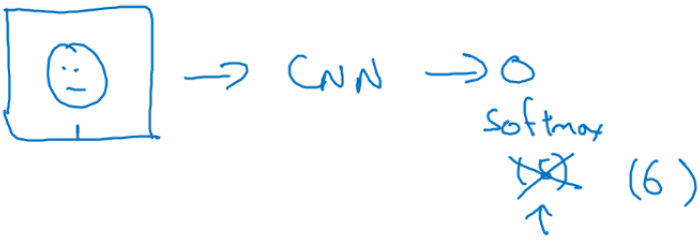
Face Recognition

- **Verification**: Input image and name/ID; output whether the input image is of the claimed person (1:1 problem)
- **Recognition**: Has a database of K persons, get an input image, output ID if the image is any of the K persons (or "not recognized") (1:K problem)
- Verification is a building block towards recognition

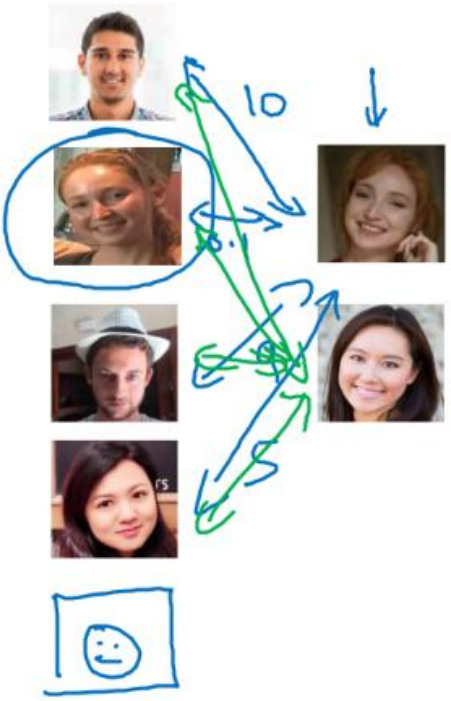
One-Shot Learning

- Learning from one example to recognize the person again
    - Softmax output for each of the people and none of the above doesn't work well. Not enough data, and if a new person joins you have to retrain ConvNet



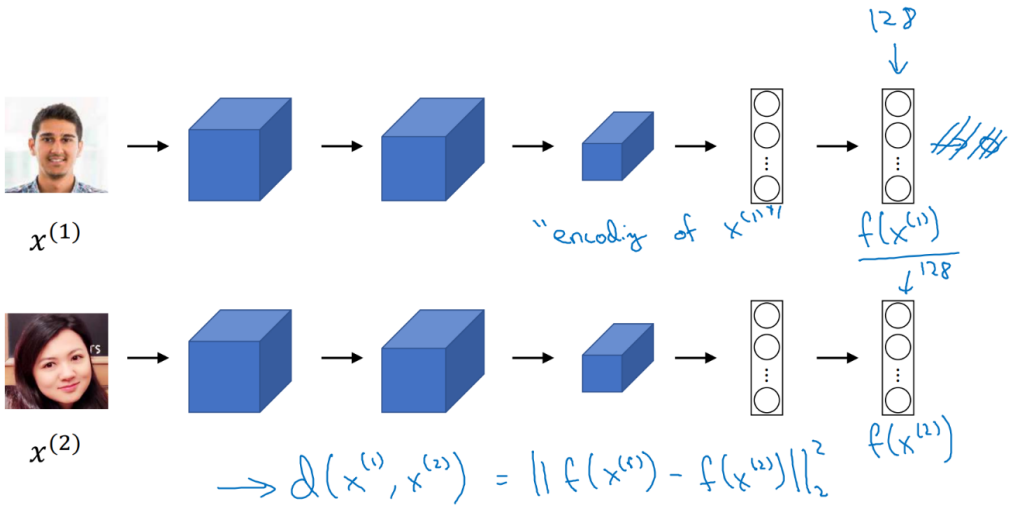


- Learning a **"similarity" function**
    - d(img1,img2) = degree of difference between images
    - d(img1,img2) ≤ τ → same person
    - d(img1,img2) > τ → different people
    - Match: d small for one comparison
    - No matches: d large for all comparisons
    - If you can learn a function d that inputs a pair of images and outputs whether they're the same person, then you can add more people without retraining



Siamese Network

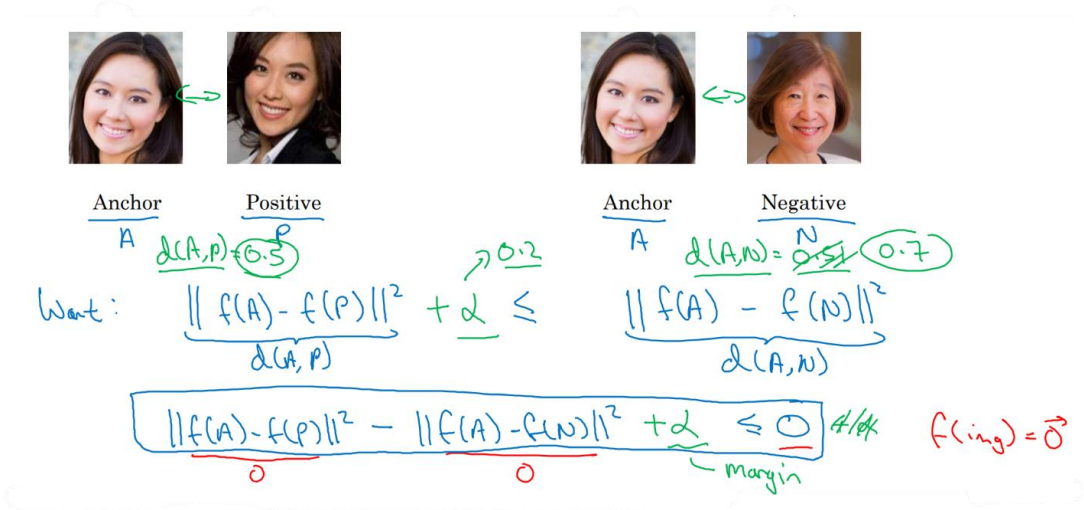
- Running two identical CNNs on two different input images and comparing them
- Ex: tell how similar to different two faces are (DeepFace)
    - $f(x^{(1)})$ = encoding of input image $x^{(1)}$ (here a vector, not softmax output)
    - Feed 2nd picture to same NN (same parameters) and get a different vector $f(x^{(2)})$
    - Distance between x1 and x2 is the norm of the difference between the encodings
        - $d(x^{(1)}- x^{(2)}) = ||f(x^{(1)}) - f(x^{(2)})||^2$
    - Train NN so that the encoding it computes results in a function d that tells you when two pictures are of the same person



- Goal of learning
    - Parameters of NN define an encoding $f(x^{(i)})$
    - Learn parameters so that:
        - If $x^{(i)}$, $x^{(j)}$ are the same person, F is small
        - If $x^{(i)}$, $x^{(j)}$ are different people, $||f(x^{(i)}) - f(x^{(j)})||^2$ is large
    - As you vary the parameters in the layers of the NN, you end up with different encodings. Use backprop to vary these parameters to make sure the conditions are satisfied

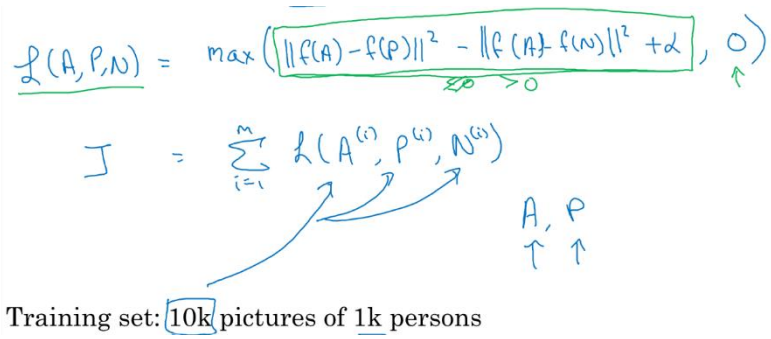
Triplet Loss

- How do you define an objective function to make a NN learn this?
- Onbe way to learn the params of the NN so that it gives out a good encoding is to define and apply gradient descent on the triplet loss function
- Learning Objective
  - Compare pairs of images. Want encodings to be similar to the same person and different for different people
  - Always look at one anchor image and measure distance. Distance between anchor and positive example (same person) small, distance between anchor and negative example (different person) large
  - Triplet: Anchor (A), Positive (P), Negative (N)
  - Want: $\|f(A) - f(P)\|^2 + \alpha \leq \|f(A) - f(N)\|^2$ [d(A,P) + α ≤ d(A< N)]
    $\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha \leq 0$
  - Adding margin α prevents it from learning trivial solutions (set everything to 0 or the same for every image). Pushes A,P and A,N pairs away from each other



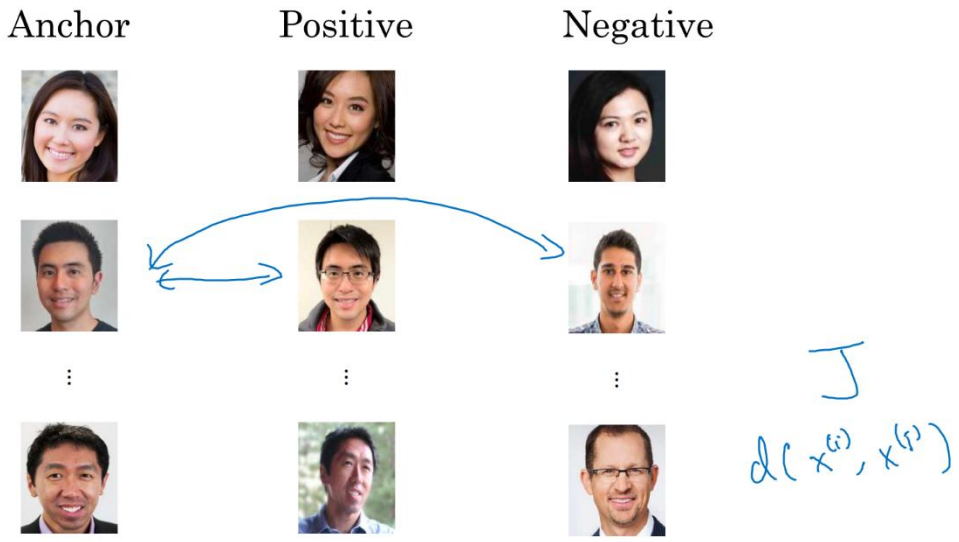
- Loss function: given 3 images A, P, N
  $L(A, P, N) = \max(\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha, 0)$ [either 0 or ≤ 0]

- Cost function: $J = \sum L(A^{(i)}, P^{(i)}, N^{(i)})$
  - Training set: 10k pictures of 1k persons (10 pictures of each), then apply with one-shot



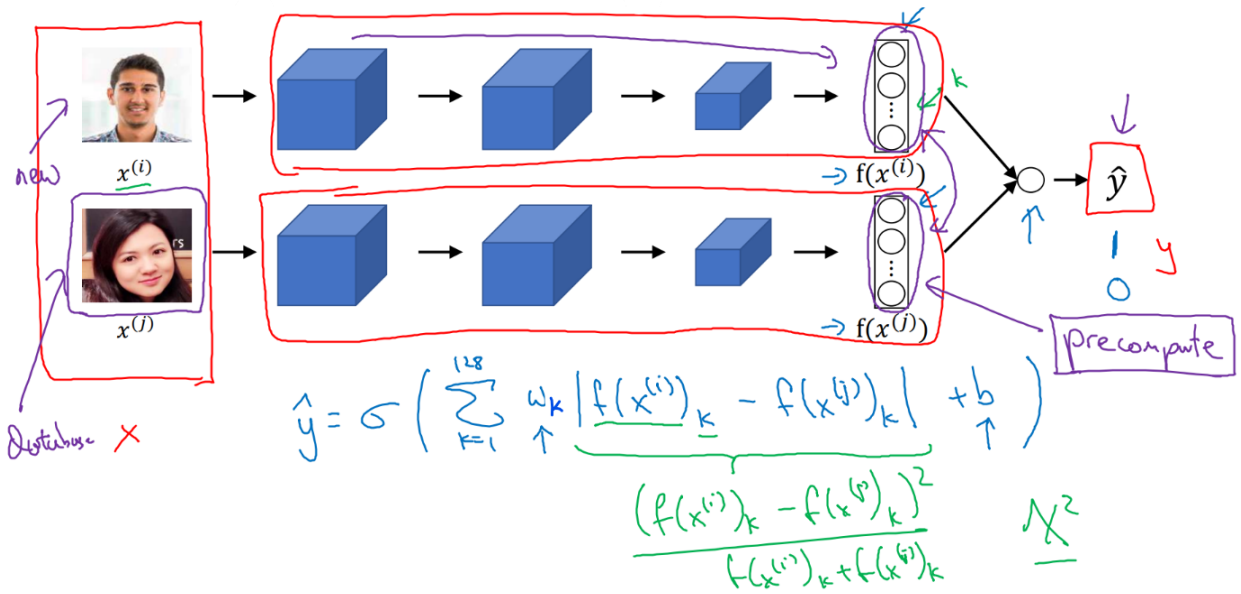
- Choosing the triplets A, P, N
  - During training, if A,P,N are chosen randomly, d(A, P) + α ≤ d(A, N) is easily satisfied [$\|f(A) - f(P)\|^2 + \alpha \leq \|f(A) - f(N)\|^2$]
  - Choose triplets that are "hard" to train on [d(A, P) + α ≈ d(A, N)]. Gradient descent has to do some work
  - Increases computational efficiency

- Training set using triplet loss
  - GD to minimize cost function → backpropagating to all NN parameters to learn an encoding such that d of two images will be small when they're of the same person and large when images of different people

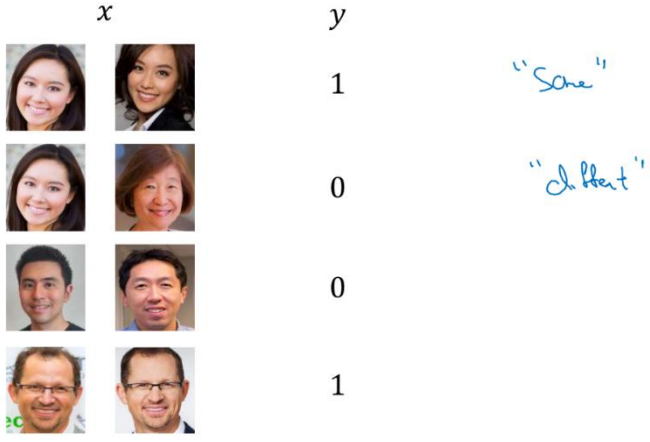


Face Verification and Binary Classification

- Triplet loss is one good way to learn the parameters of a ConvNet for face recognition, but it can also be posed as a straight binary classification problem
- Learning the similarity function
    - Have pair of NNs compute embeddings and have these be the input to a logistic regression unit to make prediction (where the target output is 1 if same person, 0 if different people). This final logistic regression unit outputs a sigmoid function applied to differences in encodings (element-wise abs value or chi-squared similarity) treated as features
    - Input is training image pair → Siamese network → output is 0 or 1
    - Precompute embedding so that when new employee walks in, other ConvNet computes encoding and compares to pre-computed to make prediction ^y

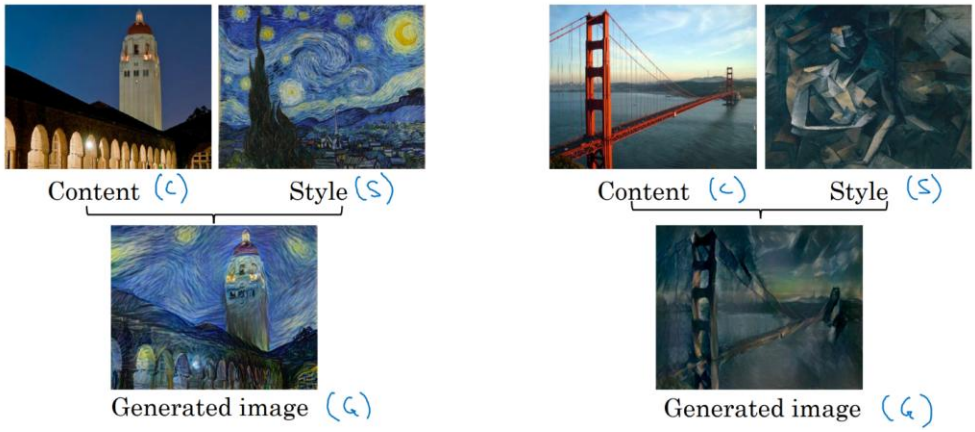


- Face verification supervised learning
    - Create training set of pairs of images, where target label is 1 when same and 0 when different
    - Use different pairs to train Siamese network using backpropagation



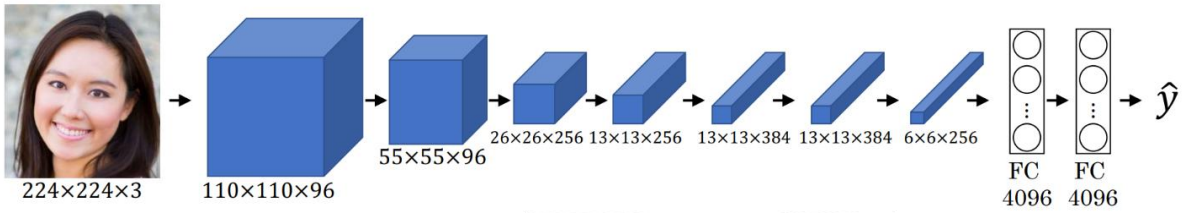
Neural Style Transfer

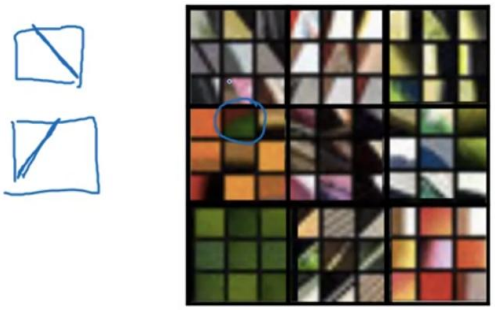
- Generate new image (G) of content (C) in the a style (S)
- Need to look at the features extracted by the ConvNet at various layers



What Are Deep ConvNets Learning?

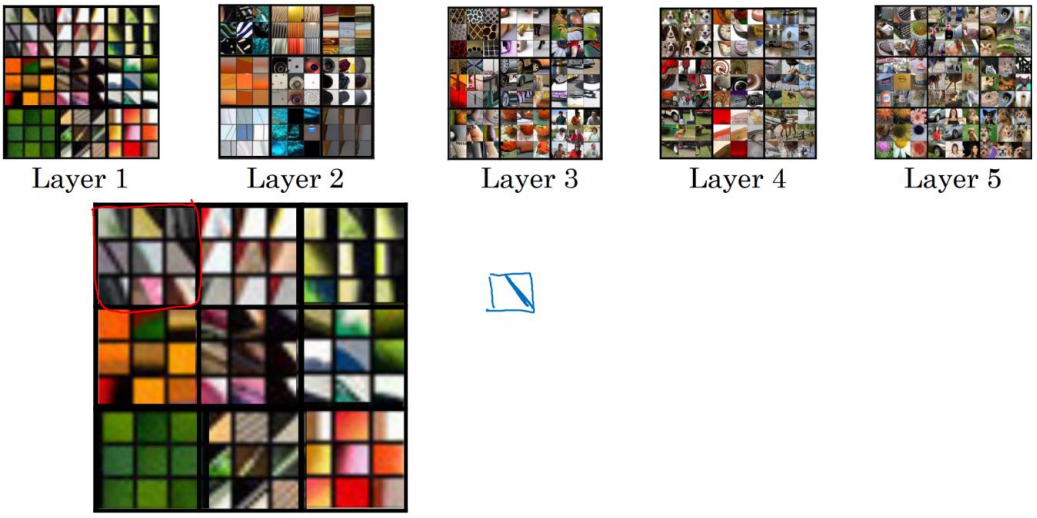
- Visualizing what a deep network is learning
  - Pick a unit in layer 1, find the nine image patches that maximize the unit's activation; repeat for other units
  - Looks for simple features like edges or color
  - In deeper layers, a hidden unit will see a larger region of the image



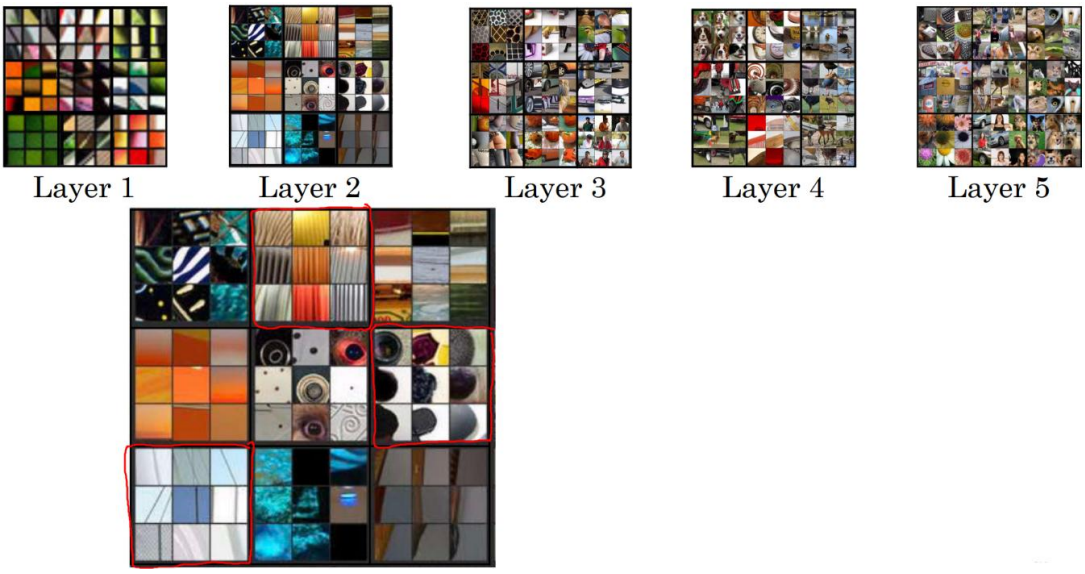


- Visualizing deep layers
  - 9 hidden patches that cause one hidden unit to be highly activated. Each grouping of 9 is a different set of nine image patches that cause one hidden unit to be activated

    - Layer 1: edges/colors



    - Layer 2: basic shapes (vertical texture, round shape on left, thin vertical line)



- Layer 3: more complex features (round in lower left, people, honeycomb shapes)

- Layer 4: even more complex (dogs, spirals, water, bird legs)

- Layer 5: sophisticated things (more varied dogs, keyboards/dots, text, flowers)

Neural Style Transfer Cost Function

- Generated Image Cost Function
  - **$J(G) = \alpha J_{content}(C, G) + \beta J_{style}(S, G)$**
  - $J_{content}(C, G)$ measures how similar the contents of the generated image are to the content image C
  - $J_{style}(S, G)$ measures how similar the style of the generated image is to the style of image S
  - α and β are hyperparameters that specify relative weighting between the content cost and style cost


Content C


Style S

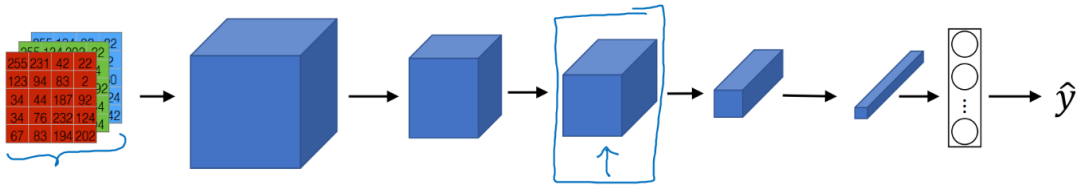
→


Generated image G

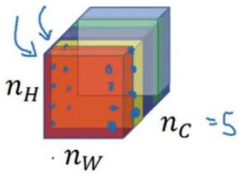
  - Find the generated image G
    - Initiate G randomly (e.g. G: 100×100×3)
    - Use gradient descent to minimize J(G)
      - G := G - ∂/∂G J(G)

- Content Cost Function
  - $J(G) = \alpha J_{content}(C, G) + \beta J_{style}(S, G)$
  - Say you use hidden layer l (typically middle) to compute content cost
  - Use pre-trained ConvNet (e.g., VGG network)
  - Let $a^{[l](C)}$ and $a^{[l](G)}$ be the activation of layer l on the images
  - If $a^{[l](C)}$ and $a^{[l](G)}$ are similar, both images have similar content
  - **$J_{content}(C, G) = ½ || a^{[l](C)} - a^{[l](G)} ||^2$** (element-wise sum of squares of differences, normalization constant optional)
  - Incentivizes algorithm to find an image G such that the hidden layer activations are similar to what to got for the content image

- Style Cost Function
  - Meaning of the "style" of an image

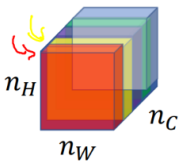


    - Say you are using layer l's activation to measure "style." Define **style** as correlation between activations across channels
    - How correlated are the activations across different channels?

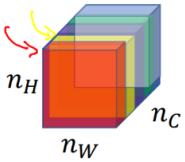


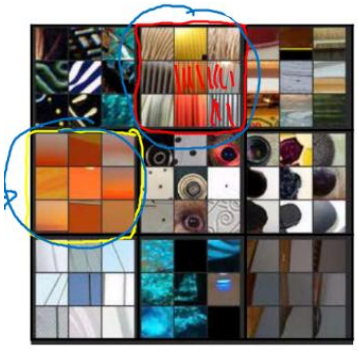
  - Intuition about style of an image
    - Let's say red channel corresponds to the 2nd neuron detecting vertical texture and the yellow channel corresponds to the 4th neuron detecting orange patches
    - Correlated: whatever part of the image has the subtle vertical texture, that part of the image will probably have an orange tint
    - Uncorrelated: whenever there is vertical texture, probably not orange
    - Correlations tells which high level texture components tend to occur or not occur together in part of an image
    - Degree of correlation between channels is a measure of style
    - Measure the degree to which in your generated image, the first channel is correlated or uncorrelated with the second channel. This tells you in the generated image how often the vert texture occurs/doesn't with orange

Style image

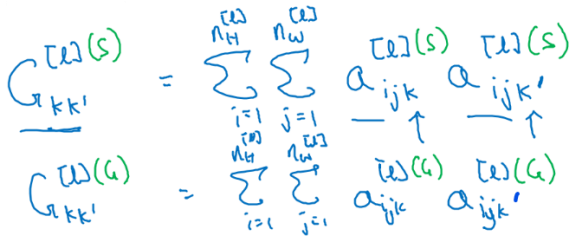


Generated Image

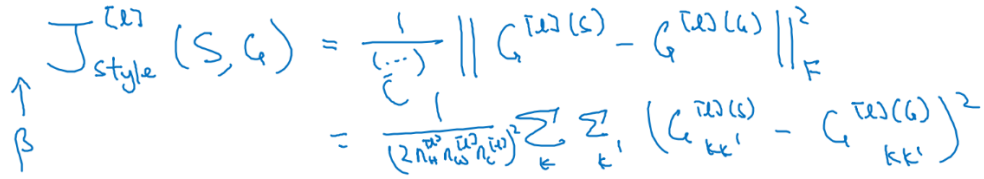




- Style matrix
  - Let $a^{[l]}_{i,j,k}$ = activation at (I, j, k) [H, W, C indeces]. $G^{[l]}$ is $n_c^{[l]}$ x $n_c^{[l]}$
  - $G^{[l]}_{kk'}$ measures how correlated the activations in channel k are compared to the activations in channel k', where k and k' range from 1 to the number of channels in that layer
  - $G^{[l]}_{kk'}$ sums over the different positions of the image (H and W) and multiplying the activations of channels k and k' together (unnormalized cross-covariance)
    - If the activations are large together, G will be large
    - If uncorrelated, G might be small
    - Calculate for style image and generated image
    - Resulting style matrices are called gram matrices in linear algebra
  - Style cost function is the sum of squares of the element-wise difference between the two style matrices (with normalization constant 2 $n_H^{[l]}$ $n_W^{[l]}$ $n_C^{[l]}$ that doesn't matter much since the cost is multiplied by hyperparameter β anyway)



"Gram matrix"



- Style cost function
  - Frobenius norm
  - Weight by additional parameters $\lambda^{[l]}$
  - Use multiple layers from different parts of NN to compute style

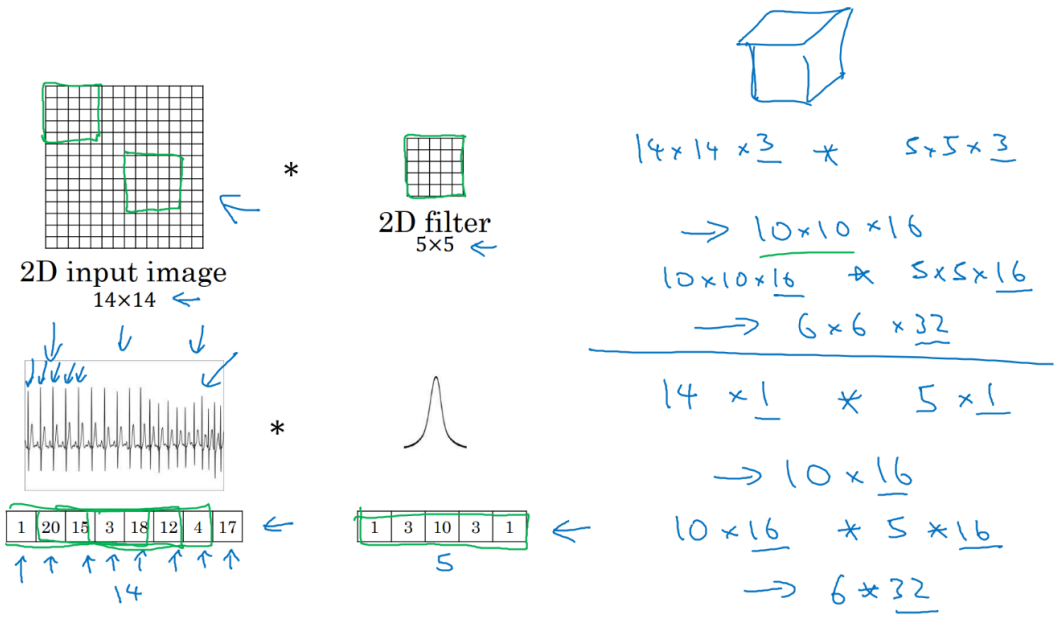
$$J_{style}^{[l]}(S,G) = \frac{1}{\left(2n_H^{[l]}n_W^{[l]}n_C^{[l]}\right)^2}\sum_k\sum_{k'}\left(G_{kk'}^{[l](S)} - G_{kk'}^{[l](G)}\right)^2$$

$$J_{style}(S,G) = \sum_l \lambda^{[l]} J_{style}^{[l]}(S,G)$$

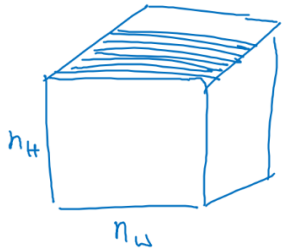
$$J(G) = \alpha J_{content}(C,G) + \beta J_{style}(S,G)$$

1D and 3D Generalization

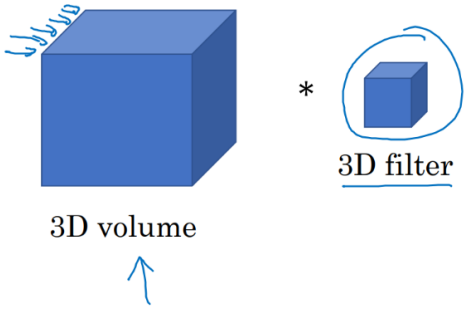
- Convolutions in 2D and 1D
  - EKG data is 1D voltage data. Want a 1D filter
  - Similarly apply 1D filter across lot of different positions throughout the 1D signal
  - Similar dimensions to 2D example (1D: 14 * 5 → 10; 2D: 14x14 * 5x5 → 10x10)
  - Usually use RNNs for 1D, but can use CNNs as well

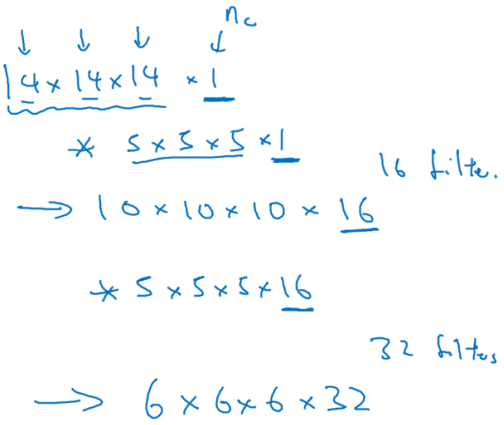


- 3D convolution
  - 3D data is made up of slices



  - Apply a 3D filter that detects features across 3D data
  - Also applies to video data, where the volume dimension is time
  - 14x14x14(x1) * 5x5x5(x1) → 10x10x10(x16)





Face verification and face recognition are the two most common names given to the task of comparing a new picture against one person's face. True/False?

False True

This is the description of face verification, but not of face recognition.

Why do we learn a function $d(img1, img2)$ for face verification? (Select all that apply.)

Given how few images we have per person, we need to apply transfer learning.

This allows us to learn to predict a person's identity using a softmax output unit, where the number of classes equals the number of persons in the database plus 1 (for the final "not in database" class).

We need to solve a one-shot learning problem.

This allows us to learn to recognize a new person given just a single image of that person.

You want to build a system that receives a person's face picture and determines if the person is inside a workgroup. You have pictures of all the faces of the people currently in the workgroup, but some members might leave, and some new members might be added. To train a system to solve this problem using the triplet loss you get many persons and take several pictures of each one. Which of the following do you agree with? (Select the best answer.)

You take several pictures of the same person because this way you can get more pictures to train the network efficiently since you already have the person in place.

You shouldn't use persons outside the workgroup you are interested in because that might create a high variance in your model.

It would be best to increase the number of persons in the dataset by taking only one picture of each person to have a more representative set of the population.

You take several pictures of the same person to train $d(\mathbf{img}_1, \mathbf{img}_2)$ using the triplet loss.

In the triplet loss: $\max(\| f(A) - f(P) \|^2 - \| f(A) - f(N) \|^2 + \alpha, 0)$. Which of the following are true about the triplet loss? Choose all that apply.

We want that $\| \boldsymbol{f}(\boldsymbol{A}) - \boldsymbol{f}(\boldsymbol{P}) \|^2 < \| \boldsymbol{f}(\boldsymbol{A}) - \boldsymbol{f}(\boldsymbol{N}) \|^2$ so the negative images are further away from the anchor than the positive images.

Being a positive image, the encoding of P should be close to the encoding of A.

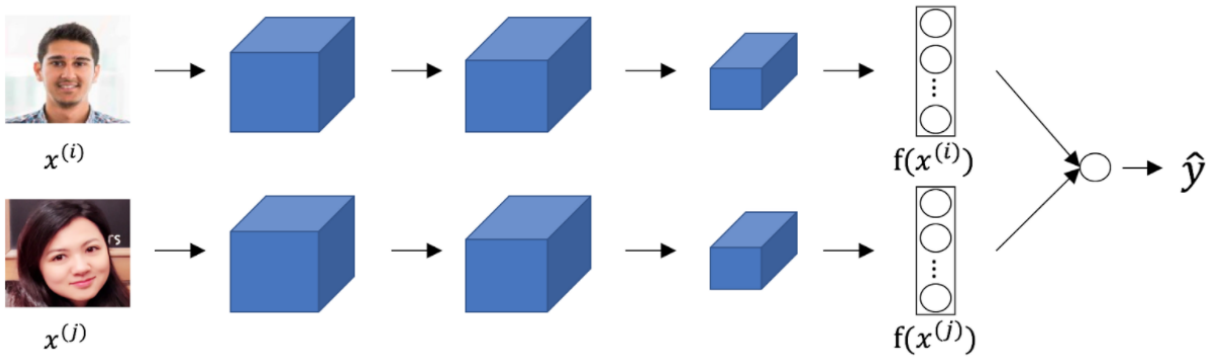
A, the anchor image, is a hyperparameter of the Siamese network.

α is a trainable parameter of the Siamese network.

$\boldsymbol{f}(\boldsymbol{A})$ represents the encoding of the Anchor.

f represents the network that is in charge of creating the encoding of the images, and A represents the anchor image.

Consider the following Siamese network architecture:



The upper and lower neural networks have different input images, but have exactly the same parameters.

True False

Parameters are shared

You train a ConvNet on a dataset with 100 different classes. You wonder if you can find a hidden unit which responds strongly to pictures of cats. (i.e., a neuron so that, of all the input/training images that strongly activate that neuron, the majority are cat pictures.) You are more likely to find this unit in layer 4 of the network than in layer 1.

True False

Neurons that understand more complex shapes are in deeper layers

Neural style transfer is trained as a supervised learning task in which the goal is to input two images (x), and train a network to output a new, synthesized image (y).

True **False**

Neural style transfer is about training the pixels of an image to make it look artistic, it is not learning any parameters.

In neural style transfer, we define style as:

$\| a^{[l](S)} - a^{[l](G)} \|^2$ the distance between the activation of the style image and the content image.

The correlation between the activation of the content image C and the style image S.

The correlation between activations across channels of an image.

correlation: $G_{kk'}^{[l](I)}$ for image I

The correlation between the generated image G and the style image S.

In neural style transfer, what is updated in each iteration of the optimization algorithm?

The pixel values of the content image C

The pixel values of the generated image G

The neural network parameters

The regularization parameters

Neural style transfer doesn't learn any parameters. It learns the pixels of an image

You are working with 3D data. The input "image" has size $64 \times 64 \times 64 \times 3$, if you apply a convolutional layer with 16 filters of size $4 \times 4 \times 4$, zero padding and stride 2. What is the size of the output volume?

$61 \times 61 \times 61 \times 14$

$64 \times 64 \times 64 \times 3$

$31 \times 31 \times 31 \times 3$

$\mathbf{31 \times 31 \times 31 \times 16}$

$n^{[l]} = [(n^{[l-1]} - f + 2p) / s] + 1 = [(64 - 4 + 2*0)] / 2] + 1 = 31$ Filters = 16